

AMERICAN INTERNATIONAL UNIVERSITY–BANGLADESH

Faculty of Science and Technology (FST)

Department of Data Science

LABORATORY MANUAL

Programming for Data Science

BDS3106

Course code and title:	BDS3106: Programming for Data Science
Academic session:	Fall 2026–2027
Faculty:	Faculty of Science and Technology
Department:	Department of Data Science

Contents

Lab 01: Python Environment and Fundamentals **1**

Lab 02: Python Collections, Loops, and Functions **4**

Lab 03: Introduction to Pandas for Data Science **6**

Lab 01

Lab 01

Python Environment, Notebooks, and Fundamentals

Lab Objective
Set up the Python programming environment, become familiar with VS Code and Google Colab, and practise variables, data types, operators, input/output, and arithmetic expressions.

Learning Outcomes

By the end of this lab, students should be able to:

- Verify a Python installation and run a Python file.
- Write and execute programs using VS Code.
- Create and run a Google Colab notebook.
- Use code and Markdown cells.
- Work with variables, data types, input/output, and arithmetic expressions.

1. Python Installation

Install Python locally. Google Colab can be used for notebook activities without installing Jupyter locally.

Windows – PowerShell

1. Download Python 3 from https://www.python.org/downloads/.
2. Run the installer and select **Add python.exe to PATH**.
3. Select **Install Now**.
4. Open PowerShell and verify the installation.

```
python --version
```

Create a file named `hello.py`:

```
print("Hello, Programming for Data Science!")
```

Navigate to the file's directory and run:

```
cd Desktop
python hello.py
```

macOS – Terminal

Install Python 3 from the official website, open Terminal, and verify the installation.

```
python3 --version
```

If Homebrew is already installed, you may alternatively use:

```
brew install python
```

Troubleshooting

If Windows reports that Python is not recognized, check whether Python was added to `PATH`. Modify or repair the installation if necessary.

2. VS Code with Python

VS Code is a code editor for writing, running, and organizing Python programs.

1. Install VS Code from https://code.visualstudio.com/.
2. Install the Python extension from the Extensions panel.
3. Create or open a course folder.
4. Create `hello.py` and enter the following code.
5. Select **Run Python File**.

```
name = input("Enter your name: ")
print("Hello,", name)
```

Typical workflow: Open folder → Create file → Write code → Run program → Check output.

3. Jupyter Notebook and Google Colab

A notebook combines explanatory text, code, and output in one document, making it useful for data processing and analysis.

1. Open Google Colab: https://colab.research.google.com/.
2. Sign in with your Google account.
3. Select **New Notebook**.
4. Rename the notebook `Lab01_Python_Basics`.

Enter and execute the following code cell using **Shift + Enter**.

```
print("Hello, Programming for Data Science!")

x = 10
y = 20
print(x + y)
```

Expected output for the arithmetic expression:

```
30
```

A code cell executes Python instructions, whereas a Markdown cell adds headings and explanations.

> **Important**
>
> Run cells in the correct order when they depend on variables created in earlier cells. Restarting the notebook session clears variables stored in memory.

4. Guided Practice

Activity 01 – Variables

Create variables to store a student's name, student ID, age, CGPA, and department. Print all five values.

```
student_name = "Rahim"
student_id = "24-12345-1"
age = 21
cgpa = 3.50
department = "Data Science"

print(student_name)
print(student_id)
```

```
print(age)
print(cgpa)
print(department)
```

Activity 02 – Data Types

Create a string, integer, float, and Boolean value. Use `type()` to display their data types.

```
name = "Rahim"
age = 21
cgpa = 3.50
is_student = True

print(type(name))
print(type(age))
print(type(cgpa))
print(type(is_student))
```

Activity 03 – User Input

Ask the user for a name, age, and department, then display a short introduction.

```
name = input("Enter your name: ")
age = input("Enter your age: ")
department = input("Enter your department: ")

print("Hello", name)
print("You are", age, "years old.")
print("You study", department + ".")
```

Activity 04 – Basic Arithmetic

Create two numbers and calculate their sum, difference, product, quotient, and remainder.

```
a = 25
b = 4

print("Sum:", a + b)
print("Difference:", a - b)
print("Product:", a * b)
print("Division:", a / b)
print("Remainder:", a % b)
```

5. List of Problems

Problem 1. Write a program that inputs two numbers and displays their sum, difference, product, and quotient. Convert the inputs to numeric values before calculating.

Problem 2. Input a student's name, student ID, age, department, and CGPA. Display the information as a formatted student profile.

Problem 3. Input a temperature in Celsius and convert it to Fahrenheit and Kelvin using the formulas:

$$F = (C \times 9/5) + 32, \qquad K = C + 273.15$$

Problem 4. Input a student's name and five course marks. Calculate the total, average, highest mark, lowest mark, and number of passed courses. Display all results. A mark of 50 or higher is considered a pass.

Lab 02

Lab 02

Core Python Collections, Loops, and Functions

Lab Objective
Practise representing and processing small datasets using strings, lists, tuples, and dictionaries. Use loops to process collections and functions to make code reusable.

Learning Outcomes

By the end of this lab, students should be able to:

- Work with strings, lists, tuples, and dictionaries.
- Access and process values stored in collections.
- Use `for` and `while` loops.
- Apply conditions inside loops.
- Write and call functions with parameters and return values.

1. Lab Environment

Use Google Colab, Jupyter Notebook, or VS Code.

Create one notebook or Python file. Add a short Markdown heading before each exercise and submit the completed work to the GitHub repository specified by the instructor.

2. Quick Python Practice

Review the following Python structures before attempting the exercises.

Structure	Example	Purpose
String	`name = "Rahim"`	Text data
List	`marks = [75, 82, 68, 91]`	Ordered, mutable collection
Tuple	`point = (10, 20)`	Ordered, immutable collection
Dictionary	`student = {"name": "Rahim"}`	Key-value data
For loop	`for mark in marks:`	Process each item
Function	`def average(values):`	Reusable code

Reminder

List indices start at zero. Tuples cannot be modified after creation. Dictionary values are accessed using their keys. Check that a collection is not empty before calculating its average.

3. Practice Problems

Problem 1 – Course Marks. Create a list containing the marks of five courses. Display the total, average, highest mark, and lowest mark. Count the number of passed courses, using 50 as the pass mark.

Problem 2 – Weekend Temperatures. Create a tuple containing six temperature readings: three for Saturday and three for Sunday. Use a `for` loop to display each reading and calculate the highest and lowest temperatures.

Problem 3 – Word Analyzer. Ask the user to enter a word or short sentence. Display the number of characters, vowels (`a`, `e`, `i`, `o`, `u`), and spaces. Use a `for` loop to examine each character.

Problem 4 – Grade Calculator. Write a function named `calculate_grade(marks)` that receives a list of marks, calculates the average, and returns the corresponding grade.

Average	Grade
80 or higher	A
70 to below 80	B
60 to below 70	C
50 to below 60	D
Below 50	F

Call the function using a list of five marks.

Problem 5 – Shopping Cart Processor. Create a list of dictionaries representing four products. Each dictionary must contain a product name, price, and quantity.
Use a loop to calculate the total cost. Write a function named `apply_discount(total)` that applies a 10% discount when the total is at least 2,000. Otherwise, return the original total.

Problem 6 – Countdown. Write a program using a `while` loop that asks the user for a non-negative integer and prints a countdown from that number to zero.

4. Suggested Data for Exercises 7–8

Use the following student records for the student-record analysis exercise.

```
students = [
    {"name": "Rahim", "department": "CSE",
     "marks": 85, "attendance": 92},
    {"name": "Karim", "department": "CSE",
     "marks": 48, "attendance": 76},
    {"name": "Sara", "department": "DS",
     "marks": 91, "attendance": 95},
    {"name": "Nadia", "department": "DS",
     "marks": 67, "attendance": 84},
    {"name": "Hasan", "department": "CSE",
     "marks": 73, "attendance": 78}
]
```

Problem 7 – Student Record Analyzer. Using the student records above, display each student's name and marks. Count the students who passed (marks ≥ 50), calculate the class average, and identify the student with the highest mark.

Use the following product records for the inventory exercise.

```
products = [
    {"name": "Notebook", "price": 120, "quantity": 3},
    {"name": "Pen", "price": 20, "quantity": 5},
    {"name": "USB Drive", "price": 850, "quantity": 2},
    {"name": "Mouse", "price": 650, "quantity": 1}
]
```

Problem 8 – Product Inventory Summary. Calculate each product's line total using `price * quantity`. Calculate the total inventory value and identify the product with the highest line total.

Lab 03 (Final-Term)

Lab 03

Introduction to Pandas for Data Science

Lab Objective
Understand the fundamentals of Pandas and use its core data structures to load, inspect, select, filter, sort, and perform basic analysis on a real-world dataset.

Learning Outcomes

By the end of this lab, students should be able to:

- Understand Pandas Series and DataFrame.
- Load datasets from CSV files.
- Inspect dataset structure and data types.
- Select and filter rows and columns.
- Sort data and perform basic aggregations.

1. Laboratory Environment and Dataset

Required Libraries and Tools

- Python 3.x
- Pandas
- Jupyter Notebook or Google Colab

Dataset: Titanic Passenger Dataset

Download the training dataset from the Kaggle Titanic competition:

https://www.kaggle.com/competitions/titanic/data

Use the file `train.csv` throughout this laboratory.

2. Introduction to Pandas

Pandas is a Python library used for data manipulation and analysis. It provides two fundamental data structures:

- **Series:** A one-dimensional labeled array.
- **DataFrame:** A two-dimensional tabular structure containing rows and columns.

```python
import pandas as pd

# Creating a Series
scores = pd.Series([85, 90, 75, 88])

print(scores)

# Creating a DataFrame
data = {
    "Name": ["Ayesha", "Rahim", "Nadia"],
    "Age": [21, 22, 20],
    "Score": [85, 90, 75]
}

df_example = pd.DataFrame(data)

print(df_example)
```

3. Loading and Inspecting a Dataset

Pandas can load data from external files, including CSV, Excel, and JSON formats.

```
# Load the Titanic dataset
df = pd.read_csv("train.csv")

# Display the first five rows
print(df.head())

# Display the last five rows
print(df.tail())

# Dataset dimensions
print("Shape:", df.shape)

# Column names
print(df.columns)

# Data types
print(df.dtypes)

# Dataset information
df.info()

# Descriptive statistics
print(df.describe())
```

Problem 1 – Dataset Inspection.

1. Display the first 10 rows.
2. Find the number of rows and columns.
3. Identify all column names.
4. Determine the data type of each column.
5. Display descriptive statistics for numerical columns.

4. Selecting and Filtering Data

Pandas provides different methods to access specific columns and rows. The `loc` and `iloc` methods are commonly used for row selection.

Selecting Columns

```
# Select one column
print(df["Name"])

# Select multiple columns
print(df[["Name", "Age", "Sex", "Survived"]])
```

Selecting Rows

```
# Select rows using labels
print(df.loc[0:4, ["Name", "Age"]])

# Select rows using integer positions
print(df.iloc[0:5, 0:4])
```

Filtering Data

Boolean conditions allow us to filter records based on specific criteria.

```python
# Passengers older than 30
print(df[df["Age"] > 30])

# Female passengers
print(df[df["Sex"] == "female"])

# Passengers who survived
print(df[df["Survived"] == 1])

# Female passengers who survived
print(df[
    (df["Sex"] == "female") &
    (df["Survived"] == 1)
])
```

Problem 2 – Data Filtering. Using the Titanic dataset:

1. Display passengers belonging to first class (`Pclass == 1`).
2. Find passengers younger than 18.
3. Find male passengers who survived.
4. Find passengers who paid a fare greater than 100.

5. Sorting and Aggregation

Pandas provides methods to sort records and calculate summary statistics.

Sorting Data

```python
# Sort passengers by age
print(df.sort_values(by="Age"))

# Sort by fare in descending order
print(df.sort_values(
    by="Fare",
    ascending=False
))

# Sort by passenger class and fare
print(df.sort_values(
    by=["Pclass", "Fare"]
))
```

Basic Aggregation

```python
# Average passenger age
print(df["Age"].mean())

# Median passenger fare
print(df["Fare"].median())

# Maximum and minimum fare
print(df["Fare"].max())
print(df["Fare"].min())

# Count passengers by gender
print(df["Sex"].value_counts())

# Average fare by passenger class
print(df.groupby("Pclass")["Fare"].mean())

# Survival rate by gender
print(df.groupby("Sex")["Survived"].mean())
```

Understanding Survival Rate

The `Survived` column contains binary values: 0 represents did not survive, while 1 represents survived. Therefore, calculating its mean gives the survival rate.

Problem 3 – Sorting and Aggregation. Using the Titanic dataset:

1. Display the 10 passengers who paid the highest fares.
2. Calculate the average passenger age.
3. Count the number of male and female passengers.
4. Calculate the average fare for each passenger class.
5. Calculate the survival rate for each passenger class.

6. Final Laboratory Challenge

Titanic Dataset Analysis
Using Pandas and the Titanic dataset, write a Python program to answer the following questions:

1. How many passengers are in the dataset?
2. What is the average passenger age?
3. Which passenger class has the highest number of passengers?
4. What is the average fare paid by male and female passengers?
5. Which gender has a higher survival rate?
6. How does the survival rate differ across passenger classes?

Present the results using appropriate Pandas operations and briefly explain your findings.

7. Submission Requirements

Students must submit a Notebook containing:

- Completed code for all laboratory tasks.
- Outputs of the executed code.
- Solutions to the exercises.
- Answers to the final laboratory challenge.

Submission Reminder

Ensure that all code cells execute successfully. Use meaningful variable names and provide brief explanations of the analytical results.